CSE 314 — Operating System Sessional
Online on xv6 Scheduling · Section B1

Foreground-First Round Robin

30 minutes. Fresh xv6 checkout, CPUS := 1.

The Idea

Modify the xv6 scheduler so that each cycle visits foreground processes first, then background processes. Each cycle consists of two complete process-table scans.

- A process is foreground if priority >= FG_THRESHOLD; otherwise it is background. Priority has no other effect within either class.
- First scan: visit slots from the beginning to the end of the process table and run each RUNNABLE foreground process once.
- Second scan: start again at the beginning and run each RUNNABLE background process once. Then repeat the two scans. Start the background scan even if foreground processes remain runnable.
- A selected process runs until it yields, blocks, or exits. On return, continue with the following table slot. Check state and class when visiting each slot; wakeups and priority changes do not restart a scan.

Skip sleeping and unused slots. An empty class simply completes its scan without running anything. Preserve xv6's interrupt and process-lock handling. With fixed priorities, each continuously runnable process gets one selection per cycle; foreground means earlier service, not extra CPU share.

Specification Parameters

Define the following macros in kernel/param.h:

Macro	Value	Meaning
`MIN_PRIORITY`	1	Lowest priority a process may request.
`MAX_PRIORITY`	10	Highest priority a process may request.
`DEFAULT_PRIORITY`	5	Priority every process starts with.
`FG_THRESHOLD`	5	Priority at or above this is foreground.

System Calls

```
int setpriority(int priority);
int getpriority(void);
```

setpriority sets the caller's priority and returns 0. Invalid input resets priority to DEFAULT_PRIORITY and returns −1. getpriority returns the current priority. A fork() child inherits its parent's priority. A changed priority determines the class when the scheduler next visits that process's slot.

The syscall bodies are supplied in syspriority.c; append them to kernel/sysproc.c. Add int priority to struct proc, initialize it to DEFAULT_PRIORITY on allocation and inherit it on fork. Wire the calls through kernel/syscall.h, kernel/syscall.c, user/usys.pl and user/user.h using unused syscall numbers.

Logging

Keep a toggleable log line for every selection, showing the selected process's class — a global flag or #define that you flip and rebuild is fine:

```
PICK: Process 5 (testloop) from FOREGROUND at priority 8
```

Test Program

Use the attached testloop.c as-is: save it to user/testloop.c and add $U/_testloop to UPROGS in the Makefile. Calling syntax: testloop <iterations> <priority>.

Sample Session

Three testloop processes, equal work, different priorities, launched one line at a time (scheduler logging off). The & runs each process in the background so the next can be launched while it is still running:

```
$ testloop 60 8 &
PID 4: starting 60 iterations at time 8, priority 8
$ testloop 60 4 &
PID 6: starting 60 iterations at time 15, priority 4
$ testloop 60 2 &
PID 8: starting 60 iterations at time 22, priority 2
PID 4: finished at time 107
PID 6: finished at time 121
PID 8: finished at time 123
```

This session was captured under QEMU with CPUS := 1 using the supplied testloop.c and a reference implementation. Priority 8 is foreground; priorities 4 and 2 are background. Priority changes ordering, not turn length. PIDs, timestamps and completion order vary; exact timings are not part of evaluation.

For a deterministic check, suppose the process table contains P1, P2, P3 and P4 in that order, with priorities 2, 8, 4 and 5. If all remain runnable and priorities stay fixed, each cycle selects P2, P4, P1, P3. P2 and P4 run in the foreground scan; P1 and P3 run in the background scan.

Submission

Run these commands from the xv6 checkout. Replace STUDENT_ID with your student ID and submit the patch containing your scheduling changes and test program.

```
git add --all
git diff HEAD > ../STUDENT_ID.patch
```